# Modélisation mathématique

Task Offloading Problem (IoT–Edge–Cloud–VM)

Iyad Ben Mosbah, Noussayba El Marrakchi, Tasnim Atig, Abdoulaye Mbaye
**IA B**

## 1 Introduction

Le *Task Offloading Problem* vise à déterminer **où et comment** exécuter chaque tâche issue d'un ensemble d'objets connectés (IoT). Chaque tâche peut être exécutée :

— localement sur l'appareil IoT,
— sur une **machine virtuelle (VM)** d'un nœud **Edge/Fog**,
— ou sur une VM hébergée dans le **Cloud**.

Chaque option implique un compromis entre :

— la **latence** (rapidité d'exécution),
— la **consommation énergétique**,
— et le **coût d'exécution**.

**Contexte :** dans un système de **caméras intelligentes**, chaque caméra capture et traite des images en temps réel. Elle peut décider :

— de traiter les images localement (faible latence mais forte énergie),
— d'envoyer le flux à un serveur Edge pour un prétraitement rapide,
— ou de déléguer le traitement complet au Cloud pour un calcul plus précis.

Cette problématique s'inscrit dans le cadre de l'**optimisation multi-objective (MOO)** où les trois critères précédents sont minimisés simultanément.

Le modèle présenté ici s'inspire de :

— **Peng et al. (2021)** : formulation multi-objective contrainte Edge–Cloud ;
— **Keshavarznejad et al. (2021)** : modèles de latence et d'énergie adaptés aux environnements Fog ;
— **Adhikari et al. (2022)** : intégration de la granularité VM et du coût réseau ;
— **Djemai (2021)** : cadre architectural IoT–Fog–Cloud et contraintes de QoS.

## 2 Variables de décision

$$x_{ijk} = \begin{cases} 1, & \text{si la tâche } i \text{ est exécutée sur la VM } k \text{ du nœud } j, \\ 0, & \text{sinon.} \end{cases}$$

Chaque tâche est affectée à une seule machine virtuelle :

$$\sum_{j \in \mathcal{N}} \sum_{k \in \mathcal{K}_j} x_{ijk} = 1, \quad \forall i \in \mathcal{T}.$$

où :

— $\mathcal{N}$ : ensemble des nœuds (IoT, Edge, Cloud),
— $\mathcal{K}_j$ : ensemble des VMs disponibles sur le nœud $j$.

La section suivante présente les paramètres techniques et économiques utilisés dans le modèle.

# 3 Paramètres du modèle

| Symbole | Description | Exemple |
|---|---|---|
| $p_i$ | charge de calcul de la tâche $i$ (cycles) | $2 \times 10^9$ |
| $d_i$ | taille des données (MB) | 10 |
| $s_{jk}$ | vitesse CPU de la VM $(j,k)$ (cycles/s) | Cloud > Edge > IoT |
| $b_{ij}$ | bande passante entre IoT et $j$ (MB/s) | 5–20 |
| $r_{ij}$ | latence réseau (s) | 0.01 (Edge), 0.1 (Cloud) |
| $\kappa_{jk}^{proc}$ | énergie par cycle sur $(j,k)$ (J/cycle) | $10^{-10}$ |
| $\kappa^{tx}$ | énergie d'émission (J/MB) | 0.5 |
| $c_{jk}^{cpu}$ | coût CPU sur $(j,k)$ (€/s) | $10^{-5}$–$5 \times 10^{-5}$ |
| $c_j^{net}$ | coût réseau (€/MB) | $10^{-3}$ |
| $a_i$ | délai maximal autorisé (s) | 1–5 |
| $C_{jk}^{cpu}$ | capacité CPU totale (cycles) | – |
| $C_j^{net}$ | capacité réseau maximale (MB) | – |

# 4 Calcul des grandeurs élémentaires

## Latence pour une frame i $L_{ijk}$

$$L_{ijk} = \begin{cases} \dfrac{p_i}{s_{jk}}, & \text{si } j = \text{IoT}, \\ \dfrac{d_i}{b_{ij}} + \dfrac{p_i}{s_{jk}} + r_{ij}, & \text{sinon.} \end{cases}$$

## Énergie pour une frame i $E_{ijk}$

$$E_{ijk} = \begin{cases} \kappa_{jk}^{proc}\, p_i, & \text{si } j = \text{IoT}, \\ \kappa^{tx}\, d_i + \kappa_{jk}^{proc}\, p_i, & \text{sinon.} \end{cases}$$

## Coût pour une frame i $C_{ijk}$

$$C_{ijk} = c_{jk}^{cpu}\, \frac{p_i}{s_{jk}} + c_j^{net}\, d_i \cdot \mathbf{1}_{j \neq \text{IoT}}.$$

# 5 Contraintes

$$\sum_i p_i x_{ijk} \leq C_{jk}^{cpu}, \qquad \forall j, k, \tag{1}$$

$$\sum_i d_i x_{ijk} \leq C_j^{net}, \qquad \forall j \neq \text{IoT}, \tag{2}$$

$$\sum_{j,k} x_{ijk} L_{ijk} \leq a_i, \qquad \forall i, \tag{3}$$

$$x_{ijk} \in \{0, 1\}. \tag{4}$$

# 6 Objectifs

$$\text{Minimiser } f_1(x) = \sum_{i,j,k} x_{ijk} L_{ijk} \qquad \text{(latence totale)} \tag{5}$$

$$\text{Minimiser } f_2(x) = \sum_{i,j,k} x_{ijk} E_{ijk} \qquad \text{(énergie totale)} \tag{6}$$

$$\text{Minimiser } f_3(x) = \sum_{i,j,k} x_{ijk} C_{ijk} \qquad \text{(coût global)} \tag{7}$$

Le problème complet d'optimisation s'écrit donc :

$$\min_x (f_1(x), f_2(x), f_3(x))$$

sous les contraintes précédentes :

$$\sum_{j,k} x_{ijk} = 1, \quad x_{ijk} \in \{0, 1\}.$$

# 7 Metaheuristic Selection

La résolution du problème d'optimisation multi-objective formulé dans la section précédente nécessite l'utilisation d'une métaheuristique capable d'explorer efficacement un espace de recherche discret, tout en approximant un ensemble de solutions non dominées (front de Pareto). Parmi les approches existantes, l'algorithme NSGA-II, pourtant très répandu en optimisation multi-objective, est explicitement interdit dans le cadre de ce projet. Il est donc nécessaire d'identifier une alternative robuste, adaptée à la nature binaire du vecteur de décision $x_{ijk}$ et aux contraintes de notre environnement IoT–Edge–Cloud virtualisé.

Après analyse des principales familles de métaheuristiques (Cuckoo Search, Dragonfly Algorithm, Ant Colony Optimization, Harmony Search, etc.), nous retenons le **Multi-Objective Firefly Algorithm (MOFA)** comme la solution la plus pertinente pour ce travail.

## 7.1 Principe général du MOFA

Le problème d'offloading IoT–Edge–Cloud étudié est formulé comme un problème d'optimisation multi-objectifs visant à minimiser simultanément la latence, la consommation énergétique

et le coût d'exécution. Ces objectifs étant antagonistes, aucune solution unique ne peut être optimale, ce qui justifie la recherche d'un front de Pareto.

Le *Multi-Objective Firefly Algorithm* (MOFA) est une extension multi-objective de l'algorithme Firefly, inspirée du comportement collectif des lucioles. Chaque solution candidate est assimilée à une luciole, dont l'attractivité dépend de ses performances multi-objectives. Une solution est attirée vers une autre si celle-ci la domine au sens de Pareto.

Le choix du MOFA repose sur plusieurs motivations :

— sa simplicité conceptuelle et sa bonne adaptabilité aux problèmes discrets,
— sa capacité à maintenir un ensemble de solutions non dominées,
— sa compatibilité naturelle avec une évaluation approximée via un modèle surrogate,
— son efficacité pour explorer des espaces de décision groupés.

Contrairement à des algorithmes évolutionnaires plus complexes, le MOFA permet un contrôle fin du compromis entre exploration et exploitation, ce qui est particulièrement adapté au contexte de décisions d'offloading par groupes de trames vidéo.

# 8 Implémentation de l'approche proposée

Cette section décrit de manière détaillée l'implémentation de l'approche d'optimisation multi-objectifs proposée pour le problème d'offloading de tâches dans une architecture IoT–Edge–Cloud. L'implémentation repose sur (i) une modélisation réaliste des tâches et des ressources, (ii) une approximation par modèle *surrogate* afin de réduire le coût des évaluations, et (iii) une optimisation hybride combinant métaheuristique multi-objectifs et raffinement local par apprentissage.

## 8.1 Étape 1 — Construction du dataset à partir d'une vidéo réelle

Le dataset utilisé dans ce travail est construit à partir d'une vidéo réelle, *Big Buck Bunny*, choisie comme support représentatif d'un flux vidéo continu tel qu'on peut en rencontrer dans des applications IoT (vidéosurveillance, analyse embarquée, streaming intelligent).

La vidéo est découpée en trames (frames) compressées au format JPEG afin de simuler un prétraitement effectué sur l'équipement IoT. Chaque trame est ensuite modélisée comme une tâche élémentaire $\mathcal{T}_i$, caractérisée par :

— une taille de données $d_i$ (en MB), correspondant à la taille du fichier compressé,
— une charge de calcul $p_i$ (en cycles CPU), définie par $p_i = \alpha_i \cdot d_i$,
— une deadline $a_i$ (en secondes), représentant la contrainte temporelle maximale admissible.

**Variabilité des tâches et ajustements de cohérence.** Un point critique observé au début du projet est que des trames consécutives issues d'une même vidéo peuvent être très similaires (absence de changement de plan, faible mouvement). Cela conduit à des tâches presque identiques et peut faire émerger artificiellement une solution dominante (front de Pareto pauvre).

Pour limiter ce biais, les tailles des trames extraites sont d'abord stockées dans une liste, puis les tâches sont construites en sélectionnant aléatoirement des trames dans cette liste, plutôt que d'utiliser des frames consécutives. Ainsi, deux tâches successives ne correspondent pas nécessairement à des images proches dans le temps, ce qui introduit une hétérogénéité réelle.

Deux mécanismes complémentaires sont ensuite intégrés :

— **Hétérogénéité de charge CPU :** $\alpha_i$ est tiré aléatoirement dans un intervalle réaliste (cycles/MB), permettant de modéliser des traitements de complexité variable.
— **Contraintes temporelles réalistes :** les deadlines $a_i$ sont choisies dans un intervalle court (environ 30 à 90 ms), afin de forcer un compromis réel entre latence, énergie et coût.

**Décision groupée.** Les trames sont ensuite regroupées en paquets de taille fixe (*groupes*). Chaque groupe constitue une **unité de décision d'offloading**. Cette stratégie permet de réduire fortement la dimension du problème (au lieu de décider trame par trame), tout en conservant une cohérence temporelle (un groupe représente un court segment du flux).

## 8.2 Étape 2 — Définition des ressources IoT–Edge–Cloud et encodage des solutions

Les ressources de calcul sont modélisées sous forme de profils (IoT, Edge, Cloud), chacun étant caractérisé par :

— une puissance CPU (cycles/s),
— une bande passante réseau (MB/s),
— une latence fixe (s),
— un coût énergétique de calcul (J/cycle) et de transmission (J/MB),
— un coût économique du calcul et du réseau.

Les options (VMs) sont **aplaties** en une liste indexée, et une solution d'offloading correspond à l'affectation de chaque groupe à une option.

**Encodage one-hot.** Une solution est encodée sous forme d'un vecteur one-hot : pour chaque groupe, un unique choix parmi les options possibles est activé. Cet encodage rend la solution compatible avec l'apprentissage du surrogate et les opérateurs discrets des métaheuristiques.

## 8.3 Étape 3 — Fonction d'évaluation réelle (Latence, Énergie, Coût)

La fonction d'évaluation réelle calcule précisément les trois objectifs **latence**, **énergie** et **coût** pour une solution donnée.

**Latence.** Pour chaque tâche, la latence combine :

— le temps de calcul $p_i/s_{\text{cpu}}$,
— le temps de transmission $d_i/b_{\text{net}}$ (Edge/Cloud),
— une latence fixe liée au type de ressource.

**Pénalisation des deadlines.** Une pénalisation non linéaire est appliquée si $L_i > a_i$. Cette pénalité est volontairement **modérée mais non linéaire** : l'objectif est de forcer des arbitrages, sans rendre le problème dégénéré (où une seule stratégie domine).

**Modélisation réaliste du Cloud.** Sans correction, le Cloud peut dominer artificiellement grâce à sa puissance CPU et sa bande passante. Pour éviter ce biais tout en conservant un Cloud *utile*, le modèle intègre :

— un **surcoût économique** spécifique (premium),
— une **congestion** dépendant de la charge (impactant la latence),
— des paramètres énergétiques réseau distincts.

Ainsi, le Cloud devient intéressant pour certaines tâches (celles où la réduction de latence compense le coût), sans être systématiquement optimal.

**Contrôle de cohérence par stratégies homogènes.** Avant toute optimisation, des stratégies « tout IoT », « tout Edge » et « tout Cloud » sont évaluées pour vérifier l'ordre de grandeur des objectifs. Par exemple, on observe typiquement :

— IoT : latence plus élevée mais énergie et coût faibles,

— Edge : latence faible/modérée mais énergie/coût plus élevés,
— Cloud : latence compétitive, mais énergie/coût pénalisés par la transmission et le premium.

## 8.4 Étape 4 — Modèle surrogate (Autoencoder + MLP)

L'évaluation exacte étant coûteuse (latence/énergie/coût calculés sur toutes les tâches), un modèle surrogate est introduit afin d'approximer rapidement les objectifs.

Le surrogate est composé de deux modules :
— un **autoencoder** chargé de projeter les décisions one-hot (dimension élevée) vers un espace latent continu (dimension réduite),
— un **MLP** qui prédit les trois objectifs (latence, énergie, coût) à partir de la représentation latente.

**Entraînement.** Un ensemble de solutions aléatoires est généré, évalué par la fonction réelle, puis utilisé pour l'apprentissage. Les objectifs sont normalisés par min–max pour stabiliser l'entraînement. L'autoencoder est d'abord entraîné en reconstruction, puis l'encodeur est affiné conjointement avec le MLP en prédiction.

L'intérêt principal est de **réduire drastiquement** le nombre d'évaluations réelles pendant l'optimisation, tout en conservant une précision suffisante pour guider l'exploration vers des zones prometteuses.

## 8.5 Étape 5 — Optimisation par MOFA avec archive Pareto vraie

L'optimisation multi-objectifs est réalisée via le *Multi-Objective Firefly Algorithm* (MOFA). Chaque luciole correspond à une solution d'affectation des groupes.

**Mouvement discret.** Une luciole $x_i$ est attirée vers $x_j$ si $x_j$ domine $x_i$ (au sens de Pareto). Le mouvement est adapté au caractère discret : au lieu d'un déplacement continu, il consiste à **copier** certains choix de groupes de $x_j$ vers $x_i$ avec une probabilité contrôlée.

**Archive et validation réelle.** L'optimisation se fait principalement dans l'espace du surrogate. Cependant, pour garantir la validité finale, une **archive Pareto vraie** est maintenue :
— périodiquement, un sous-ensemble de solutions prometteuses est réévalué par la fonction réelle,
— ces solutions sont fusionnées dans l'archive, puis filtrées par dominance réelle,
— l'archive représente le front final réellement non dominé.

Ce mécanisme hybride combine exploration rapide (surrogate) et fiabilité (validation exacte ciblée). Dans les expériences réalisées, le MOFA obtient par exemple un front non dominé de taille 26.

## 8.6 Étape 6 — Raffinement par Q-learning et comparaison avec MOPSO

### 8.6.1 Raffinement local par Q-learning

Afin d'améliorer localement le front obtenu par MOFA, une phase de raffinement par Q-learning simplifié est appliquée. L'idée est d'effectuer une optimisation fine sur les solutions déjà bonnes, plutôt que de relancer une recherche globale.

**Définition des états et actions.**

- **États :** une solution non dominée de l'archive MOFA (front réel).
- **Actions :** tenter d'améliorer localement un objectif spécifique (latence, énergie ou coût) en modifiant un petit nombre de groupes.

**Reward et politique.** Le reward est défini comme l'amélioration d'un score multi-objectifs normalisé. Une politique $\varepsilon$-greedy est utilisée pour équilibrer exploration (et éviter de rester bloqué) et exploitation (amélioration des solutions déjà bonnes).

**Impact observé.** Cette étape densifie fortement le front : dans les résultats obtenus, le nombre de solutions non dominées passe typiquement de 26 (MOFA) à 44 (MOFA+RL). L'hypervolume augmente également de manière significative (par exemple de 0.8485 à 1.0181), ce qui indique une meilleure couverture de l'espace des objectifs.

### 8.6.2 Comparaison avec une métaheuristique de référence : MOPSO

Le *Multi-Objective Particle Swarm Optimization* (MOPSO) est utilisé comme méthode de comparaison. Une version allégée est implémentée pour le contexte discret.

**Principe (adapté au discret).** Les particules représentent des solutions. À chaque itération :

- un leader est sélectionné dans une archive Pareto,
- certaines décisions de groupes sont copiées depuis le leader (exploration guidée),
- une mutation aléatoire est appliquée sur quelques groupes (diversité).

**Lecture comparative.** Dans les expérimentations, MOPSO produit un front très dense (par exemple 77 solutions non dominées) et un spacing faible (ex. 0.0132), signe d'une distribution serrée. Cependant, cette densité est obtenue au prix d'un nombre important d'évaluations exactes. En comparaison, MOFA+RL obtient un front plus compact, mais de grande qualité globale (hypervolume supérieur dans nos résultats : 1.0181 contre 0.9651 pour MOPSO).

**Synthèse.** L'approche hybride MOFA + raffinement Q-learning offre un compromis robuste :

- exploration rapide via surrogate,
- validation exacte ciblée via archive vraie,
- amélioration locale efficace via Q-learning,
- performances compétitives voire supérieures à MOPSO sur l'hypervolume.

# 9 Évaluation et analyse des performances

Cette section présente l'évaluation quantitative et qualitative des performances des différentes approches considérées. L'analyse repose sur des indicateurs classiques de l'optimisation multi-objectifs, ainsi que sur une interprétation fine des fronts de Pareto obtenus et des stratégies d'offloading associées.

## 9.1 Qualité des fronts de Pareto obtenus

Les fronts de Pareto issus de la baseline, du MOFA, du MOFA raffiné par Q-learning (MOFA+RL) et du MOPSO sont comparés dans l'espace des objectifs (latence, énergie, coût).

La Figure correspondante montre clairement que les solutions générées s'organisent selon un compromis croissant entre rapidité, consommation énergétique et coût économique.

La baseline génère un nuage relativement dispersé de solutions, dont une partie seulement est non dominée. Les fronts obtenus par les approches d'optimisation présentent une structure plus régulière et une meilleure couverture de l'espace objectif, en particulier pour MOFA+RL et MOPSO.

## 9.2 Nombre de solutions non dominées

Le nombre de solutions non dominées (ND) constitue un premier indicateur de la richesse du front :

— Baseline ND : 25 solutions,
— MOFA : 26 solutions,
— MOFA+RL : 44 solutions,
— MOPSO : 77 solutions.

Le MOFA seul permet déjà d'atteindre un front comparable à la baseline, tout en utilisant un nombre beaucoup plus réduit d'évaluations exactes. L'ajout du raffinement par Q-learning augmente significativement la densité du front (+69% de solutions ND par rapport à MOFA). Le MOPSO génère le front le plus dense, mais au prix d'un coût computationnel plus élevé.

## 9.3 Indicateur d'hypervolume

L'hypervolume (HV) mesure le volume de l'espace objectif dominé par un front de Pareto par rapport à un point de référence. Il permet d'évaluer simultanément la qualité et l'étendue des compromis proposés.

Les hypervolumes obtenus sont :

— Baseline ND : $HV \approx 0.8370$,
— MOFA : $HV \approx 0.8485$,
— MOFA+RL : $HV \approx 1.0181$,
— MOPSO : $HV \approx 0.9651$.

Ces résultats montrent que :

— le MOFA améliore légèrement la couverture de l'espace objectif par rapport à la baseline,
— le raffinement par Q-learning apporte un gain majeur en hypervolume, indiquant des compromis globalement plus performants,
— bien que MOPSO génère davantage de solutions, son hypervolume reste inférieur à celui de MOFA+RL, suggérant une couverture moins efficace des zones les plus intéressantes de l'espace objectif.

## 9.4 Métrique de spacing

La métrique de spacing évalue l'uniformité de la distribution des solutions sur le front de Pareto. Une valeur faible indique une meilleure régularité.

Les valeurs obtenues sont :

— Baseline ND : 0.0553,
— MOFA : 0.0657,
— MOFA+RL : 0.0497,
— MOPSO : 0.0132.

Le MOPSO présente le spacing le plus faible, ce qui reflète un front très dense et régulièrement réparti. Le MOFA+RL améliore nettement la régularité par rapport au MOFA seul et à la baseline, tout en conservant une diversité suffisante des solutions. Le spacing légèrement plus élevé du MOFA est cohérent avec un front plus compact mais moins dense.

## 9.5 Analyse des compromis latence, énergie et coût

L'analyse statistique (min, médiane, moyenne) des objectifs sur les fronts non dominés met en évidence des comportements distincts :

- la baseline présente des solutions moyennes correctes, mais peu de solutions extrêmes réellement intéressantes,
- le MOFA améliore progressivement les compromis sans favoriser un objectif unique,
- le MOFA+RL permet d'atteindre des solutions à très faible énergie et coût (énergie moyenne $\approx 0.000498$, coût moyen $\approx 0.000041$), au prix d'une légère augmentation de la latence,
- le MOPSO offre de très bonnes latences moyennes, mais avec une dispersion plus large des compromis coût/énergie.

Ces résultats confirment que l'approche proposée ne cherche pas à optimiser un objectif unique, mais à fournir un ensemble de solutions équilibrées adaptées à différents scénarios applicatifs.

## 9.6 Lien avec les stratégies d'offloading

L'analyse des répartitions de tâches par type de ressource montre une différence claire entre les approches :

- la stratégie gloutonne privilégie fortement Edge et Cloud, en fonction de la latence immédiate,
- le MOFA et le MOFA+RL exploitent davantage l'IoT tout en déléguant certaines tâches spécifiques à l'Edge ou au Cloud,
- le Cloud est utilisé de manière ciblée, principalement pour des tâches où le gain de latence justifie le surcoût énergétique et économique.

Ce comportement confirme que les ajustements apportés au modèle (hétérogénéité des tâches, pénalisation modérée du Cloud, congestion) permettent d'obtenir des stratégies d'offloading réalistes et non triviales.

## 9.7 Synthèse de l'évaluation

L'évaluation met en évidence que :

- le MOFA permet d'obtenir rapidement un front pertinent avec un nombre limité d'évaluations exactes,
- le raffinement par Q-learning améliore significativement la qualité globale des compromis (hypervolume maximal),
- le MOPSO reste une référence robuste pour générer des fronts très denses, mais au prix d'un coût computationnel plus élevé.

Ainsi, l'approche hybride proposée (MOFA + surrogate + Q-learning) constitue un excellent compromis entre efficacité, qualité des solutions et réalisme opérationnel dans un contexte IoT–Edge–Cloud.

# 10 Conclusion

Ce projet a permis de modéliser le *Task Offloading Problem* dans une architecture IoT–Edge–Cloud virtualisée, en cherchant à minimiser simultanément la **latence**, l'**énergie** et le **coût**. Le modèle prend en compte des éléments réalistes (transmission réseau, latences fixes, pénalité de dépassement de deadline, surcoût et congestion côté Cloud), ce qui évite les solu-

tions « trop parfaites » et force de vrais compromis.

Pour résoudre le problème, nous avons proposé une approche hybride : un *surrogate* (*Autoencoder + MLP*) pour accélérer les évaluations, un **MOFA** pour explorer efficacement l'espace de recherche, puis un raffinement par **Q-learning** pour améliorer localement les solutions déjà prometteuses. La décision par groupes de trames a aussi joué un rôle important, en réduisant la dimension du problème tout en gardant une logique cohérente.

Les résultats montrent que MOFA obtient un front compétitif par rapport à la baseline, et que le raffinement par Q-learning améliore nettement la qualité du front (densité et hypervolume). La comparaison avec MOPSO confirme que celui-ci peut générer un front très dense, mais sans forcément dépasser la qualité globale obtenue par MOFA+RL, et souvent avec plus d'évaluations exactes.

Au final, l'approche proposée fournit des solutions d'offloading réalistes et interprétables : l'IoT et l'Edge sont largement exploités, et le Cloud est utilisé de façon ciblée quand cela vaut réellement le coût. En perspective, on pourrait encore améliorer le réalisme (réseau variable, files d'attente) ou tester des méthodes RL plus avancées pour une décision en ligne, mais l'objectif du projet est atteint avec une méthode à la fois efficace et claire.

# Références

## (1) Modélisation IoT–Edge–Cloud et Offloading


— Peng, X., He, Y., Chen, X. & Hu, F. (2021). *Constrained Multi-objective Optimization for IoT-enabled Computation Offloading in Edge–Cloud Environments.* IEEE Internet of Things Journal.
— Keshavarznejad, A., Ghobaei-Arani, M. & Masdari, M. (2021). *Delay-aware Optimization of Energy Consumption for Task Offloading in Fog Environments.* Cluster Computing.
— Djemai, S. (2021). *Placement et gestion de services dans une architecture IoT–Fog–Cloud.* Thèse de doctorat, Université de Lorraine.


## (2) Firefly Algorithm et version multi-objective (MOFA)


— Yang, X.-S. (2009). *Firefly Algorithms for Multimodal Optimization.* In *Stochastic Algorithms : Foundations and Applications.* Springer.
— Yang, X.-S. (2013). *Multiobjective Firefly Algorithm for Continuous Optimization.* International Journal of Bio-Inspired Computation.
— Abdel-Basset, M., Manogaran, G., Mohamed, M. & Chang, V. (2019). *A Novel Multiobjective Firefly Algorithm for Fog Computing Resource Allocation.* IEEE Access.
— Peng, Y., Zhu, L. & Wang, X. (2022). *Improved Firefly Algorithm for Multi-Objective Optimization in Edge Computing.* Applied Soft Computing.


## (3) Surrogate Models (Autoencoder + MLP)


— Zhang, X., Wu, Y. & Li, Z. (2022). *Autoencoder-Based Dimensionality Reduction for Efficient Computation Offloading in Mobile Edge Computing.* IEEE Internet of Things Journal.
— Shi, Y., Huang, F. & Zhao, L. (2023). *Deep Surrogate Models for Multi-Objective Optimization in Edge Computing.* Neurocomputing.

## (4) Reinforcement Learning pour le Refinement (Q-learning / DQN)


— Sun, H., Li, T. & Zhao, W. (2024). *Reinforcement Learning–Enhanced Evolutionary Algorithms for Efficient Edge Offloading.* IEEE Transactions on Mobile Computing.

— Ruan, Y., Chen, B. & Wu, K. (2025). *Lightweight Q-Learning for Online Edge Offloading Decisions.* IEEE Internet of Things Journal.